

Grundlagen von Kernelmodulen

Aufbau, Lebenszyklus, Build und Diagnose

2.02

Dynamische Erweiterungen im Kernel

EINSATZ

Gerätetreiber

Hardware anbinden
und kontrollieren

SUBSYSTEME

Kernel erweitern

Dateisysteme und
Netzwerkfunktionen

PRIVILEGIEN

Keine Isolation

Kernelspace bedeutet
keine Prozess-Isolation

Kernelmodule ergänzen Funktionalität, ohne den Kernel neu zu starten.

Fest eingebaut oder ladbar?

BUILT-IN

Fest im Kernel

`CONFIG_FEATURE=y`

Teil des Kernel-Images
Bereits beim Start verfügbar
Änderung erfordert einen neuen Kernel

LOADABLE KERNEL MODULE

Ladbares Modul

`CONFIG_FEATURE=m`

Eigenständige .ko-Datei
Zur Laufzeit laden und entladen
Nützlich für Treiber und Erweiterungen

Beide Varianten arbeiten mit Kernelprivilegien.

Vom Quellcode zum laufenden Modul

- | | | |
|----|-------------------|--------------------------------------|
| 01 | Entwickeln | Quellcode und Modulmetadaten anlegen |
| 02 | Übersetzen | Mit Kbuild eine .ko-Datei erzeugen |
| 03 | Laden | Prüfen, auflösen und initialisieren |
| 04 | Ausführen | Kernel-Funktionalität bereitstellen |
| 05 | Entladen | Ressourcen vollständig freigeben |

Der Lebenszyklus endet erst, wenn das Modul alle Ressourcen freigegeben hat.

Ein minimales Kernelmodul

MINIMALMODUL

```
#include <linux/init.h>
#include <linux/module.h>

static int __init demo_init(void)
{
    pr_info("demo: geladen\n");
    return 0;
}

static void __exit demo_exit(void)
{
    pr_info("demo: entfernt\n");
}

module_init(demo_init);
module_exit(demo_exit);
MODULE_LICENSE("GPL");
```

01 Einstieg

`module_init()` registriert die Initialisierungsfunktion.

02 Aufräumen

`module_exit()` gibt reservierte Ressourcen frei.

03 Diagnose

`pr_info()` schreibt in das Kernel-Log.

Erfolgreich starten, sauber beenden

MODULE_INIT

Initialisierung

Ressourcen reservieren
Schnittstellen registrieren
Bei Erfolg: return 0
Bei Fehler: negatives errno

MODULE_EXIT

Aufräumen

Asynchrone Arbeit stoppen
Schnittstellen deregistrieren
Speicher und Ressourcen freigeben
Keine aktiven Zugriffe hinterlassen

FEHLERPFAD

```
if (!buffer) return -ENOMEM;
```

Aufräumen erfolgt in umgekehrter Reihenfolge der Initialisierung.

Lizenz, Autor und Beschreibung

MODULMETADATEN

```
MODULE_LICENSE("GPL");  
MODULE_AUTHOR("Linux Kernel Training");  
MODULE_DESCRIPTION("Einfaches Trainingsmodul");  
MODULE_VERSION("1.0");
```

01 Lizenz beeinflusst APIs

GPL-exklusive Symbole sind nicht für beliebig lizenzierte Module verfügbar.

02 Metadaten prüfen

modinfo demo.ko zeigt Lizenz, Beschreibung und weitere Eigenschaften.

Metadaten dokumentieren Herkunft, Lizenz und Zweck des Moduls.

Externe Module mit Kbuild bauen

KBUILD-MAKEFILE

```
obj-m += demo.o
KDIR ?= /lib/modules/$(shell uname -r)/build
all:
    $(MAKE) -C $(KDIR) M=$(CURDIR) modules
clean:
    $(MAKE) -C $(KDIR) M=$(CURDIR) clean
```

01 obj-m

Definiert das ladbare Modul.

02 KDIR

Verweist auf den passenden Kernel-Build.

03 Ergebnis

demo.ko ist das ladbare Kernelmodul.

Eine eingerückte Zeile in einem Makefile beginnt mit einem Tab !

Build und laufender Kernel müssen passen

01 Kernelversion

uname -r zeigt den laufenden Kernel.
Header und Build-Tree müssen passen.

02 Build-Konfiguration

Architektur, Konfiguration und Symbolversionen müssen kompatibel sein.

03 Toolchain

Compiler und Kernel-Build-System müssen funktionsfähig verfügbar sein.

04 Kompatibilitätsprüfung

modinfo zeigt vermagic.
Fehler erscheinen im Kernel-Log.

Kernelversion, Build-Konfiguration und Toolchain müssen zusammenpassen.

Module laden, prüfen und entladen

- | | | |
|----|----------------------------------|--|
| 01 | <code>make</code> | Modul gegen den Kernel übersetzen |
| 02 | <code>modinfo demo.ko</code> | Metadaten und vermagic prüfen |
| 03 | <code>sudo insmod demo.ko</code> | Konkrete Moduldatei laden |
| 04 | <code>lsmod</code> | Geladene Module anzeigen |
| 05 | <code>sudo dmesg tail</code> | Kernmeldungen lesen und danach mit <code>sudo rmmod demo</code> entladen |

Laden, prüfen und entladen bilden einen kontrollierten Testzyklus.

insmod und modprobe unterscheiden

DATEIBASIERT

insmod

Lädt eine konkrete .ko-Datei.
Löst Abhängigkeiten nicht selbst auf.
Praktisch beim lokalen Testen.

SYSTEMINTEGRIERT

modprobe

Lädt installierte Module per Name.
Berücksichtigt Modulabhängigkeiten.
Verwendet den Kernel-Modul-Tree.

depmod pflegt Abhängigkeiten. Aktive Referenzen können das Entladen verhindern.

Kernelmeldungen gezielt einsetzen

LOG-LEVEL

```
pr_info("demo: Modul gestartet\n");  
pr_warn("demo: ungültiger Parameter\n");  
pr_err("demo: Initialisierung fehlgeschlagen\n");  
pr_debug("demo: interner Status=%d\n", status);
```

01 Live beobachten

`sudo dmesg -w`
verfolgt neue Kernelmeldungen.

02 Kernel-Journal

`sudo journalctl -k -f`
zeigt Kernelmeldungen
fortlaufend.

Kernelmeldungen erscheinen im Kernel-Log, nicht auf der Standardausgabe.

Modulparameter steuern das Verhalten

MODULPARAMETER

```
static int interval_ms = 1000;  
module_param(interval_ms, int, 0644);  
MODULE_PARM_DESC(interval_ms, "Intervall in Millisekunden");
```

01 Beim Laden übergeben

```
sudo insmod demo.ko  
interval_ms=500
```

02 Über sysfs auslesen

```
cat /sys/module/demo/  
parameters/interval_ms
```

Modulparameter machen Kernelcode kontrolliert konfigurierbar.

Kernel-APIs und exportierte Symbole

SYMBOL-EXPORT

```
EXPORT_SYMBOL(training_helper);  
EXPORT_SYMBOL_GPL(training_internal_helper);
```

01 Exportierte Symbole

Nur verfügbare und passende Schnittstellen können aufgelöst werden.

02 GPL-exklusive Symbole

Lizenzangaben beeinflussen den Zugriff auf bestimmte Kernel-APIs.

Kernel-APIs können sich zwischen Kernelversionen ändern.

Sicherheit beim ersten Modul

AUSWIRKUNGEN

System schützen

Kernelcode läuft privilegiert.
Fehler gefährden das Gesamtsystem.
In einer geeigneten Umgebung testen.

SICHERHEITSRICHTLINIEN

Richtlinien beachten

Signaturpflicht und Secure Boot prüfen.
Kernel-Taint und Lizenzstatus beachten.
Ladefehler systematisch analysieren.

PRAXIS

Das erste Modul vollständig durchlaufen

Schreiben, bauen, laden, prüfen und entladen

Eine isolierte Testumgebung begrenzt die Auswirkungen von Fehlern.